计算机学院

并行程序设计实验报告

Lab1 CPU 架构编程

Cache 与超标量对程序性能的影响

姓名：柯云超

学号：2413575

专业：计算机科学与技术

2026 年 3 月 27 日

摘要

本文围绕两个 CPU 架构实验问题展开：矩阵列内积中的 cache 局部性优化，以及浮点数组求和中的超标量优化。针对矩阵问题，本文实现并比较了逐列访问的平凡算法、行优先的 cache 优化算法以及 4 路循环展开版本；针对求和问题，本文实现并比较了链式累加、两路独立累加、四路独立累加与两两归约四种写法。实验采用固定公式生成测试数据，通过高精度计时与统一累计时长阈值的方法测量平均执行时间，结合工作集规模与 L2/L3 cache 大小分析性能变化。结果表明，行优先算法在大规模下可获得数量级加速（$n = 2048$ 时达 22.41 倍）；四路独立累加最高达 3.81 倍加速。VTune 与汇编分析证实，矩阵优化本质在于降低 Memory Bound，求和优化本质在于缩短依赖链提高 IPC。

关键词： cache 局部性，超标量，循环展开，高精度计时，VTune

1 实验任务

本次实验包含两个问题：

1. 给定 $n \times n$ 矩阵 B 和向量 a，计算每一列与向量的内积，比较

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i.$$

的两类设计思路：逐列访问的平凡算法与 cache 友好的行优先算法。

2. 给定 n 个浮点数，计算其和，比较

$$S = \sum_{i=0}^{n-1} x_i.$$

的两类设计思路：链式累加与超标量优化算法。

此外，实验还加入了两项进阶内容：对矩阵算法加入 4 路循环展开，对关键 workload 使用 VTune 与汇编观察分析性能原因。

2 实验平台与测试方法

2.1 实验平台

源码仓库链接：`https://github.com/kyc001/parallel`。实验平台配置如表 1 所示。

项目	配置
CPU	13th Gen Intel(R) Core(TM) i9-13900H
体系结构	x86_64, 14C/20T, 单核绑核测试
L1d / L2 / L3 Cache	48 KiB / 1280 KiB / 24 MiB
操作系统	Windows 11 Home China 25H2
编译器	MSYS2 g++ 14.2.0
编译选项	`-std=c++17 -O3 -march=native -g`
额外控制	`-fno-tree-vectorize -fno-tree-slp-vectorize -fno-omit-frame-pointer`
绑核 / Profiler	CPU 0 (P-core) / Intel VTune 2025.10.0

表 1 实验平台配置

关闭自动向量化的目的是避免编译器 SIMD 化掩盖 cache 局部性和指令级并行本身的差异；单核绑核用于减少调度抖动。

本实验使用 `-O3` 优化级别，并在可能情况下开启 `-march=native` 以利用目标平台的 SIMD 单元和指令调度特性。关闭向量化（`-fno-tree-vectorize`）是为了把变量收敛到算法本身的 cache

局部性和指令级并行，避免编译器自动生成的 SIMD 指令把两类优化效果混在一起，使得实验现象更易分析。从性能角度看，`-O0` 通常仅用作正确性验证的基线；`-O2` 会开启大部分优化但可能保留部分循环；`-O3` 额外启用激进的内联和向量化（本实验中手动关闭），更适合本实验关注的高级优化效果分析。

2.2 测试数据与规模设计

两部分实验均采用固定公式生成数据：

$$a_i = 0.5 + ((19i + 13) \bmod 127)/127.0, \quad B_{i,j} = 0.2 + ((137i + 23j) \bmod 257)/257.0.$$

矩阵实验选取 $n = \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\}$，对应工作集大小为 $8n^2$ Byte。其中 $n = 384$ 时矩阵约 1.13 MiB，逼近 L2；$n = 1536$ 时约 18 MiB，接近 L3；$n = 2048$ 时约 32 MiB，超过 L3。

求和实验选取 $n = \{2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{23}\}$，数组大小 $8n$ Byte，用于观察从 cache 友好区间进入带宽受限区间的性能变化。

2.3 计时方式

使用 `std::chrono::steady_clock` 计时。对每个”算法-规模”组合，先做 2 轮 warm-up，随后从较小重复次数开始运行，若累计时间不足 250 ms 则重复次数翻倍，直到总测时达到 250 ms，再以 $t_{\text{avg}} = t_{\text{total}}/\text{repeat}$ 作为单次平均耗时。此外，程序使用 `asm volatile` 编写的 `do_not_optimize` 和 `clobber_memory` 作为反优化屏障，防止编译器消除未使用的计算结果。

3 算法设计与编程实现

3.1 矩阵列内积

矩阵部分实现了三个版本：（1）**平凡算法**：外层按列、内层按行，跨步访问局部性差；（2）**行优先算法**：外层按行、内层按列，连续访存匹配行主布局；（3）**行优先 +4 路展开**：在行优先基础上手工展开内层循环。三种算法计算量同阶，性能差异主要来自访存模式。

Listing 1 平凡算法（逐列访问）

```
void matrix_dot_naive(const double* matrix,
    const double* vec, double* out, size_t n) {
  for (size_t col = 0; col < n; ++col) {
    double acc = 0.0;
    for (size_t row = 0; row < n; ++row)
      acc += matrix[row*n + col] * vec[row];
    out[col] = acc;
  }
}
```

Listing 2 行优先算法（cache 友好）

```
void matrix_dot_row_major(const double* matrix,
    const double* vec, double* out, size_t n) {
  std::fill(out, out+n, 0.0);
  for (size_t row = 0; row < n; ++row) {
    const double factor = vec[row];
    const double* row_ptr = matrix + row*n;
    for (size_t col = 0; col < n; ++col)
      out[col] += row_ptr[col] * factor;
  }
}
```

Listing 3 行优先 + 4 路展开

```
void matrix_dot_row_major_unrolled4(const double* matrix,
    const double* vec, double* out, size_t n) {
  std::fill(out, out+n, 0.0);
  for (size_t row = 0; row < n; ++row) {
    const double f = vec[row];
    const double* rp = matrix + row*n;
    size_t col = 0;
    for (; col + 4 <= n; col += 4) {
      out[col]   += rp[col]   * f;
      out[col+1] += rp[col+1] * f;
      out[col+2] += rp[col+2] * f;
      out[col+3] += rp[col+3] * f;
    }
    for (; col < n; ++col)
      out[col] += rp[col] * f;
  }
}
```

3.2 求和

求和部分实现了四个版本。核心思想是减少串行依赖：链式累加中每次加法都依赖上一次结果，而多路独立累加把运算分散到多个寄存器上，利于超标量并行发射。

Listing 4 链式累加

```
double sum_naive(const double* data, size_t n) {
  double sum = 0.0;
  for (size_t i = 0; i < n; ++i) sum += data[i];
  return sum;
}
```

Listing 5 四路独立累加

```
double sum_superscalar4(const double* data, size_t n) {
  double s0=0, s1=0, s2=0, s3=0;
  size_t i = 0;
  for (; i + 4 <= n; i += 4) {
    s0 += data[i];   s1 += data[i+1];
    s2 += data[i+2]; s3 += data[i+3];
  }
  double sum = (s0+s1) + (s2+s3);
  for (; i < n; ++i) sum += data[i];
  return sum;
}
```

3.2.1 两路独立累加

将依赖链拆为两条，理论上可将 FP 加法吞吐率提高到接近 2 倍。

Listing 6 两路独立累加

```
double sum_superscalar2(const double* data, size_t n) {
  double s0 = 0.0, s1 = 0.0;
  size_t i = 0;
  for (; i + 2 <= n; i += 2) {
    s0 += data[i];  s1 += data[i+1];
  }
  double sum = s0 + s1;
  for (; i < n; ++i) sum += data[i];
  return sum;
}
```

3.2.2 两两归约

递归式地将相邻元素配对求和，每轮规模减半，共 $\lceil \log_2 n \rceil$ 轮。虽然缩短了理论依赖深度，但引入了对 scratch 缓冲区的大量读写。

Listing 7 两两归约

```
double sum_pairwise(const double* data, size_t n,
                    vector<double>& scratch) {
  copy(data, data+n, scratch.begin());
  size_t active = n;
  while (active > 1) {
    size_t pairs = active / 2;
    for (size_t i = 0; i < pairs; ++i)
      scratch[i] = scratch[2*i] + scratch[2*i+1];
    if (active % 2 == 1)
      scratch[pairs] = scratch[active-1];
    active = pairs + (active % 2);
  }
  return scratch[0];
}
```

4 实验结果与分析

4.1 矩阵列内积实验

表 2 给出矩阵实验结果。加速比 1 表示" 平凡 / 行优先"，加速比 2 表示" 平凡 / 行优先 +U4"。

n	平凡/ms	行优先/ms	行优先 +U4/ms	加速比 1	加速比 2
64	0.001816	0.001784	0.001199	1.02	1.52
128	0.012733	0.007509	0.006713	1.70	1.90
256	0.053780	0.027620	0.026267	1.95	2.05
384	0.123919	0.070150	0.058875	1.77	2.10
512	0.986070	0.120951	0.108207	8.15	9.11
768	1.801332	0.269507	0.250062	6.68	7.20
1024	8.580819	0.531594	0.614811	16.14	13.96
1536	17.402294	1.544646	1.439448	11.27	12.09
2048	55.476013	2.761901	2.475906	20.09	22.41

表 2 矩阵列内积实验结果

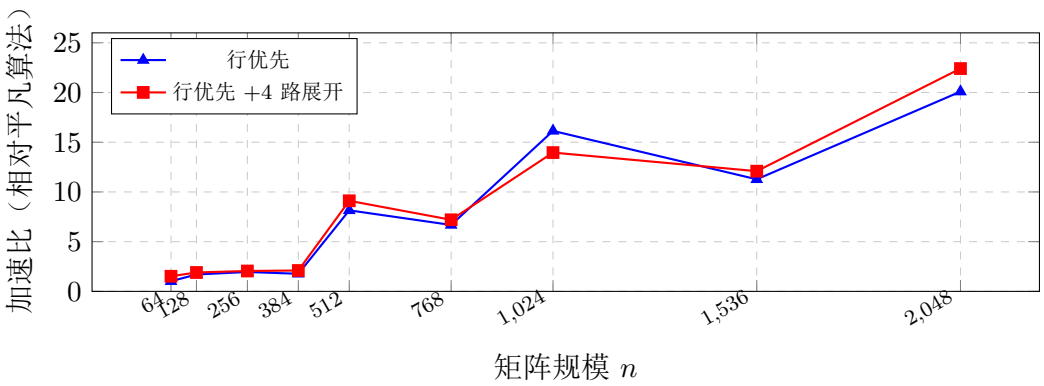

图 4.1 矩阵列内积加速比随问题规模的变化

矩阵实验有三点比较明显：

1. 当 $n \le 384$ 时（工作集 $\le$ L2），行优先带来约 1.8 ~ 2.1 倍加速，此时 cache miss 差异尚不悬殊。
2. $n = 512$ 以后，矩阵越过 L2（2 MiB），平凡算法时间急剧升高；到 $n = 512$ 时加速比已达 9.11 倍。

3. $n = 2048$ 时矩阵 32 MiB 超过 L3，平凡算法耗时 55.48 ms，行优先 + 展开仅 2.48 ms，最高 22.41 倍。

第一决定因素始终是访问顺序是否匹配行主布局；循环展开的收益建立在访存已连续化的前提之上。

4.2 求和实验

表 3 给出求和实验结果，加速比以链式累加为基准。

n	链式/ms	两路/ms	四路/ms	归约/ms	两路加速	四路加速	归约加速
1024	0.000606	0.000327	0.000171	0.000462	1.85	3.54	1.31
4096	0.002668	0.001436	0.000701	0.001938	1.86	3.81	1.38
16384	0.011021	0.005317	0.003526	0.009800	2.07	3.13	1.12
65536	0.042705	0.021633	0.014321	0.088242	1.97	2.98	0.48
262144	0.179219	0.091392	0.069757	0.395509	1.96	2.57	0.45
1048576	0.709010	0.373483	0.367339	2.759548	1.90	1.93	0.26
4194304	3.051786	2.231181	2.136283	8.038652	1.37	1.43	0.38
8388608	6.552478	4.633566	4.393747	16.510638	1.41	1.49	0.40

表 3 求和实验结果

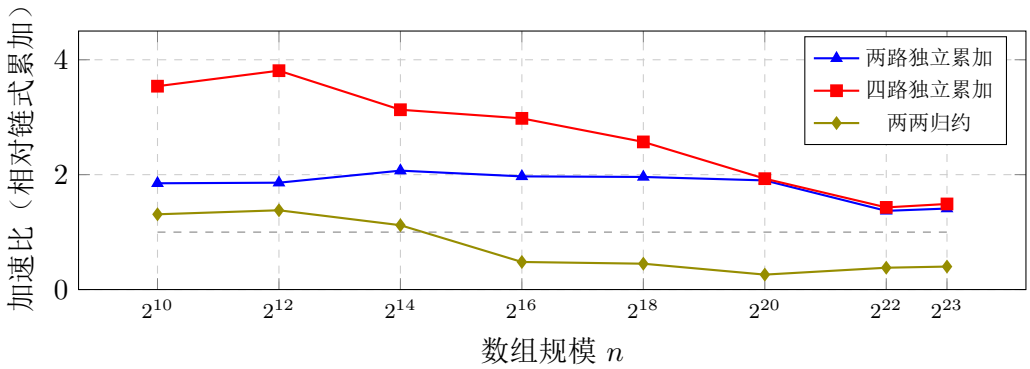

图 4.2 求和加速比随问题规模的变化

求和实验说明了超标量优化与内存约束之间的关系：

1. 两路独立累加在各规模下稳定接近 2 倍加速，说明拆为两条依赖链后处理器可并行调度。
2. 四路独立累加表现最好，$n = 4096$ 时达 3.81 倍；$n = 2^{20}$ 时仍保持 1.93 倍。
3. 当 $n \geq 2^{22}$（≥32 MiB），超过 L3，四路加速比降至 ∼1.5 倍，瓶颈转为内存带宽。
4. 两两归约在 $n \geq 2^{16}$ 后明显慢于链式累加。归约虽缩短依赖深度，但 scratch 缓冲区的反复读写带来额外访存开销。

不同求和顺序的浮点舍入差异极小，最大绝对误差约 4.66×10^{-9}。

4.3 汇编观察

通过 `objdump -Cd -Mintel` 导出各核心函数的汇编，验证编译器指令流是否符合设计预期。以下摘录关键内循环（已省略 nop 对齐）。

4.3.1 矩阵：平凡 vs 行优先

Listing 8 naive 内循环

```
vmovsd  xmm0, [r10]
vmulsd  xmm0, xmm0, [rax]
add     rax,  0x8
add     r10,  r11      ; +n*8
vaddsd  xmm1, xmm1, xmm0
jne     <loop>
```

Listing 9 row_major 内循环

```
vmovsd      xmm0, [r10+rax*8]
vfmadd213sd xmm0, xmm1,
            [rcx+rax*8]
vmovsd      [rcx+rax*8], xmm0
add         rax, 0x1
jne         <loop>
```

平凡算法 `add r10, r11` 使指针以 $n \times 8$ 字节跳跃，每次迭代跨越多个 cache line，几乎每次都 miss。行优先版本地址连续递增 8 字节，完美匹配 cache line 预取，且编译器自动生成了 FMA 指令（`vfmadd213sd`），将乘法和加法合为一条。

4.3.2 矩阵：行优先 + 4 路展开

Listing 10 unrolled4 内循环

```
vmovsd      xmm1, [r14]              ; load row[col+0]
vfmadd213sd xmm1, xmm0, [rax]       ; out[col+0] += row[col+0]*f
vmovsd      [rax], xmm1
vmovsd      xmm1, [r14-0x18]         ; load row[col+1]
vfmadd213sd xmm1, xmm0, [rax-0x18]  ; out[col+1] += ...
vmovsd      [rax-0x18], xmm1
vmovsd      xmm1, [r14-0x10]         ; load row[col+2]
vfmadd213sd xmm1, xmm0, [rax-0x10]  ; out[col+2] += ...
vmovsd      [rax-0x10], xmm1
vmovsd      xmm1, [r14-0x8]          ; load row[col+3]
vfmadd213sd xmm1, xmm0, [rax-0x8]   ; out[col+3] += ...
vmovsd      [rax-0x8], xmm1
jne         <loop>
```

每次迭代 4 条独立 FMA，步进 32 字节（0x20），覆盖半个 cache line，循环判断摊销降至 1/4。

4.3.3 求和：链式累加 vs 四路独立累加

Listing 11 链式累加（单依赖链）

```
vaddsd xmm0, xmm0, [rcx]
add    rcx, 0x10
vaddsd xmm0, xmm0, [rcx-0x8]
jne    <loop>
```

Listing 12 四路独立累加

```
vaddsd xmm0, xmm0, [rax]
vaddsd xmm3, xmm3, [rax+0x8]
add    rax, 0x20
vaddsd xmm1, xmm1, [rax-0x10]
vaddsd xmm2, xmm2, [rax-0x8]
jne    <loop>
```

链式版本虽被编译器 2 路展开，但两条 `vaddsd` 均写 `xmm0`，形成串行依赖链（FP 加法延迟 ~4 周期，IPC ~ 1/4）。四路版本使用 `xmm0`–`xmm3` 四个独立寄存器，相邻指令无数据依赖，超标量可并行发射，理论 IPC 可达约 1。

4.4 VTune 微架构分析

使用 VTune `uarch-exploration` 分析类型，对 $n = 2048$（矩阵）和 $n = 2^{20}$（求和）的关键 workload 采样。表 4 汇总了 Top-Down 方法学的核心指标。

问题	workload	CPI	Retiring %	Back-End Bound %	Memory Bound %	Core Bound %	DRAM Bound %
矩阵	`naive`	3.29	4.0	80.0	65.5	14.5	79.3
矩阵	`row_major`	0.34	45.6	50.0	36.1	13.9	17.7
矩阵	`row_major_u4`	0.49	32.1	67.4	38.0	29.4	23.1
求和	`naive`	0.83	14.5	81.8	23.6	58.2	—
求和	`superscalar2`	0.46	29.5	70.1	10.6	59.5	—
求和	`superscalar4`	0.54	27.9	75.6	27.9	47.7	—
求和	`pairwise`	0.82	19.7	78.4	60.4	18.0	—

表 4 VTune 微架构分析结果（Top-Down 指标，P-core）

矩阵问题。 平凡算法 CPI 高达 3.29，Retiring 仅 4.0%，Back-End Bound 80.0%，其中 Memory Bound 65.5%、DRAM Bound 79.3%，处理器几乎一直在等主存。改为行优先后，CPI 降至 0.34，Retiring 升至 45.6%，DRAM Bound 从 79.3% 降至 17.7%；3+ Ports Utilized 达 56.0%，多端口得到更充分利用。4 路展开 CPI 略升至 0.49（Core Bound 从 13.9% 升至 29.4%），因为展开引入了更多寄存器压力和 store 操作，但总指令数更少，实际耗时仍更短。

workload	LLC Miss Count	L1 Bound	L3 Bound	DRAM Bound
`matrix_naive`	29,712,474	0.0%	2.1%	77.7%
`row_major_u4`	13,755,775	0.0%	11.3%	21.6%

表 5 矩阵问题的 VTune memory-access 指标对比

表 5 进一步说明了 cache 层级上的变化。优化后 LLC miss 数量下降约 53.7%，同时 DRAM Bound 从 77.7% 明显降到 21.6%。虽然 L3 Bound 占比上升，但这恰好说明更多访问停留在片上 cache 内解决，没有继续落到主存，因此整体延迟显著降低。

求和问题。 链式累加 CPI 0.83，Retiring 14.5%，Core Bound 58.2%，瓶颈主要在依赖等待而非内存。四路版本将 CPI 降至 0.54，Retiring 升至 27.9%，Core Bound 降至 47.7%；同时其 Memory Bound 为 27.9%，其中 L1/L2/L3/DRAM Bound 分别为 1.9%/1.9%/24.1%/6.3%，说明大部分代价已不再是长链串行累加。两两归约则 Memory Bound 高达 60.4%（Store Bound 7.4%），验证了 scratch 缓冲区读写的高开销。

workload	L1 Bound	L2 Bound	L3 Bound	DRAM Bound
`naive`	30.6%	0.0%	3.4%	0.0%
`superscalar2`	0.0%	0.5%	5.5%	5.3%
`superscalar4`	1.9%	1.9%	24.1%	6.3%
`pairwise`	10.5%	0.0%	22.3%	28.9%

表 6 求和问题的 cache/memory 层级压力分布

表 6 可以看出，链式累加的主要问题并不是 DRAM，而是单条依赖链导致的核心等待；而两两归约由于额外的 scratch 写回，把压力明显推高到了更低效的 cache/内存层级。相比之下，超标量版本虽然仍有一定 L3/DRAM 开销，但总体上把更多周期用于有效退休指令，因此整体吞吐更高。

4.5 补充讨论

这里再补充两点实验中的观察。第一，矩阵部分在 $n = 512$ 前后出现了很明显的拐点：在这之前，行优先主要体现为稳定的 2 倍左右收益；越过 L2 以后，平凡写法的代价迅速放大，说明这部分性能差异确实主要来自访存顺序，而不是常数级的指令优化。

第二，求和部分里，两路和四路独立累加在中小规模下收益很稳定，但数组继续增大后，加速比会逐渐回落。这和 VTune 中看到的现象是一致的：依赖链被拆开以后，核心等待显著减少；但当数据规模继续扩大，内存层次的代价又会重新变得明显，因此不可能一直保持接近理想值的加速。

另外，本次测试关闭了自动向量化，并固定在 P-core 上采样。这样做是为了尽量把变量收敛到 cache 局部性和指令级并行本身，避免编译器自动生成 SIMD 后，把两类优化效果混在一起，导致实验现象不容易解释。

5 结论

本次实验的结果比较一致，可以归纳为以下几点：

1. 矩阵问题里，访存顺序是最关键的因素。行优先 + 展开在 $n = 2048$ 时达到 22.41 倍加速；VTune 中 CPI 从 3.29 降到 0.34，DRAM Bound 从 79.3% 降到 17.7%，LLC miss 也从 29.7M 降到 13.8M。
2. 求和问题里，多条独立累加链确实能缓解串行依赖。四路版本最高达到 3.81 倍加速，汇编中可以看到 4 条独立 `vaddsd` 指令链；VTune 中 CPI 从 0.83 降到 0.54，Core Bound 从 58.2% 降到 47.7%。超过 L3 后，瓶颈逐渐转为带宽，加速比也回落到 ∼1.5 倍。
3. 两两归约并不一定更快。虽然它缩短了归约深度，但 scratch 缓冲区带来的额外读写使 Memory Bound 达到 60.4%，大规模时反而慢于链式累加。

4. 从计时结果、汇编和 VTune 指标可以相互印证：矩阵优化的核心是改善 cache 局部性，求和优化的核心是缩短依赖链；循环展开只有在访存已经连续化之后才更容易带来收益。

附录：实验过程与注意事项

1. 编译时关闭自动向量化（`-fno-tree-vectorize`）以隔离 cache/ILP 效应；绑核 CPU 0（P-core）减少调度抖动。
2. 计时采用 250 ms 阈值自适应重复次数，保证各规模下均有充分采样。
3. `results/vtune_cli/` 包含 7 组 `uarch-exploration` 导出（3 矩阵 + 4 求和）；线程绑 P-core，E-core 数据为 0 属正常。
4. 正确性通过 `max_abs_diff` 校验：矩阵 bit-exact，求和最大误差 $\sim 4.66 \times 10^{-9}$。